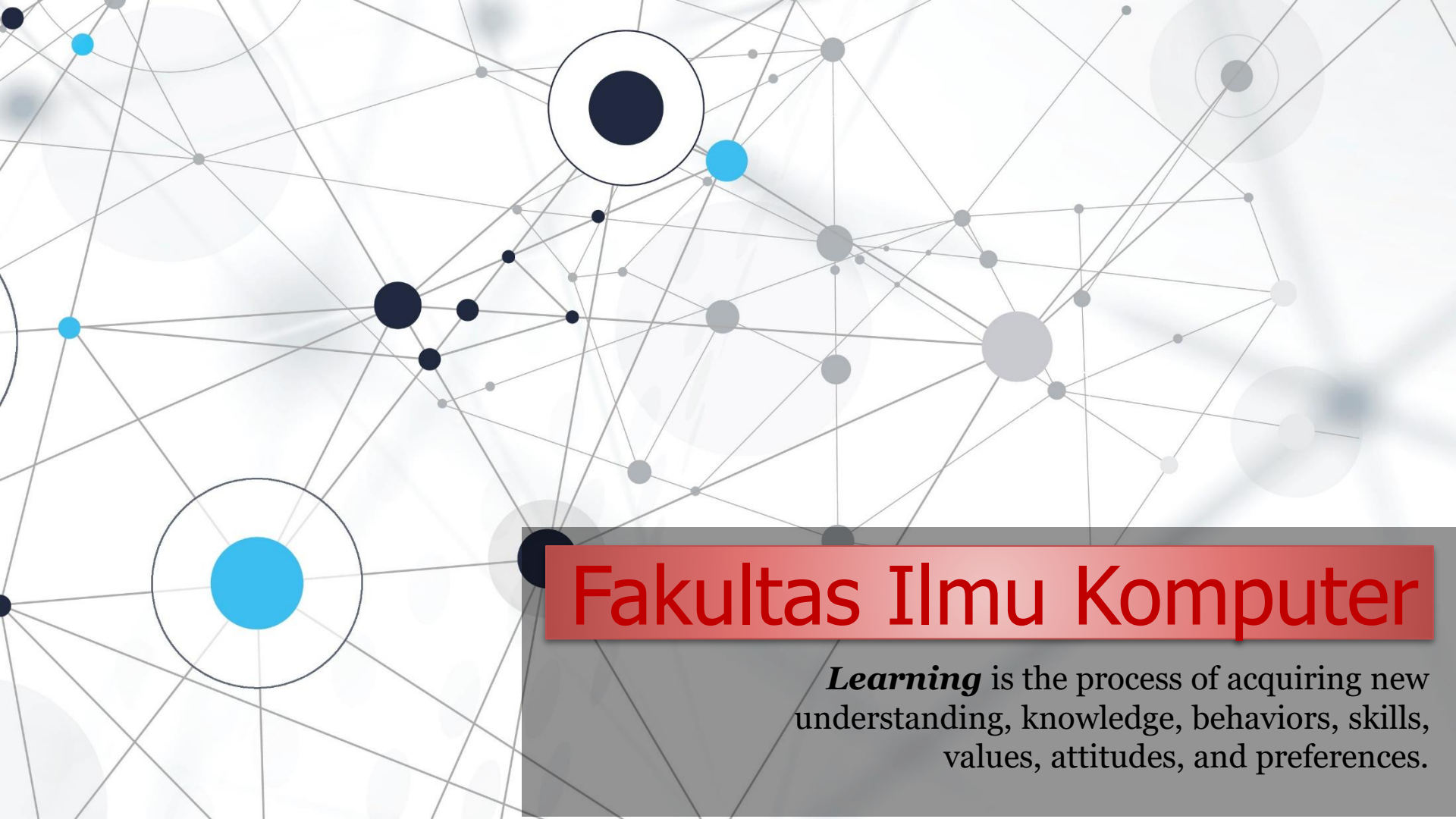


An abstract network diagram with various sized nodes (black, blue, and grey) connected by thin grey lines. Some nodes are highlighted with larger circles. The background is white with faint grey circles.

Fakultas Ilmu Komputer

Learning is the process of acquiring new understanding, knowledge, behaviors, skills, values, attitudes, and preferences.

OOP in Python – Building Blocks

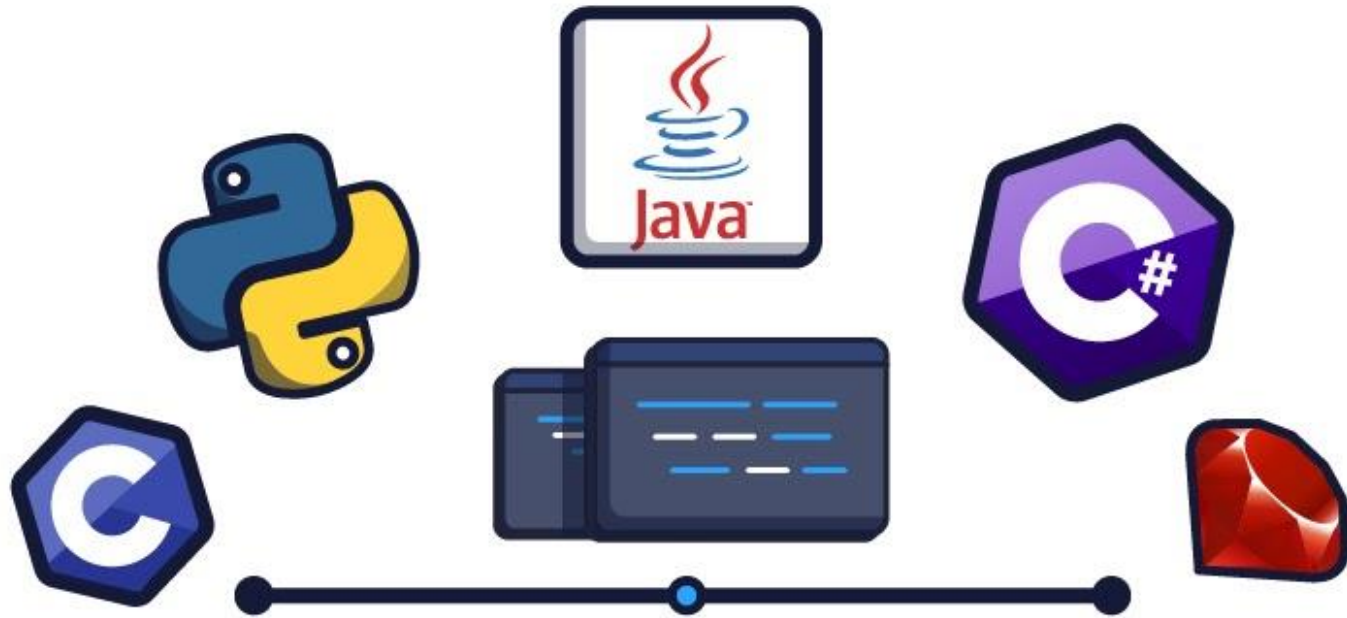


**Welcome to *the* Object Oriented
Programming *course***

4 Best Object-Oriented Programming Languages

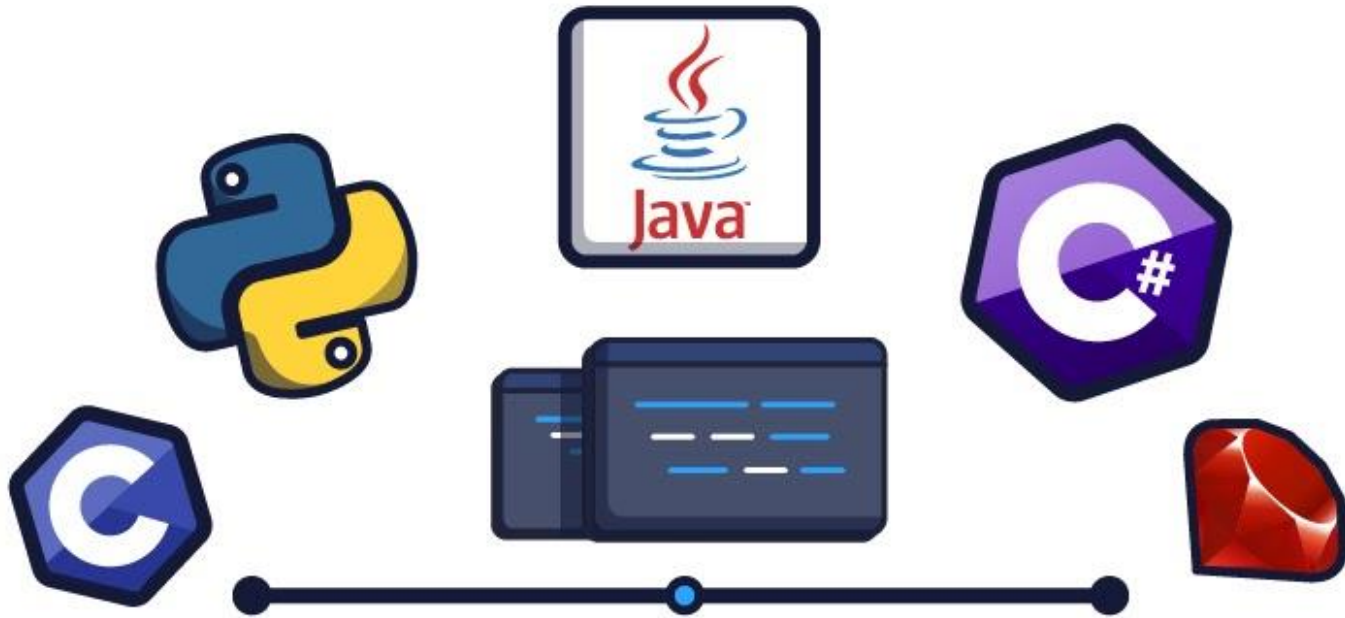


4 Best OOP Languages



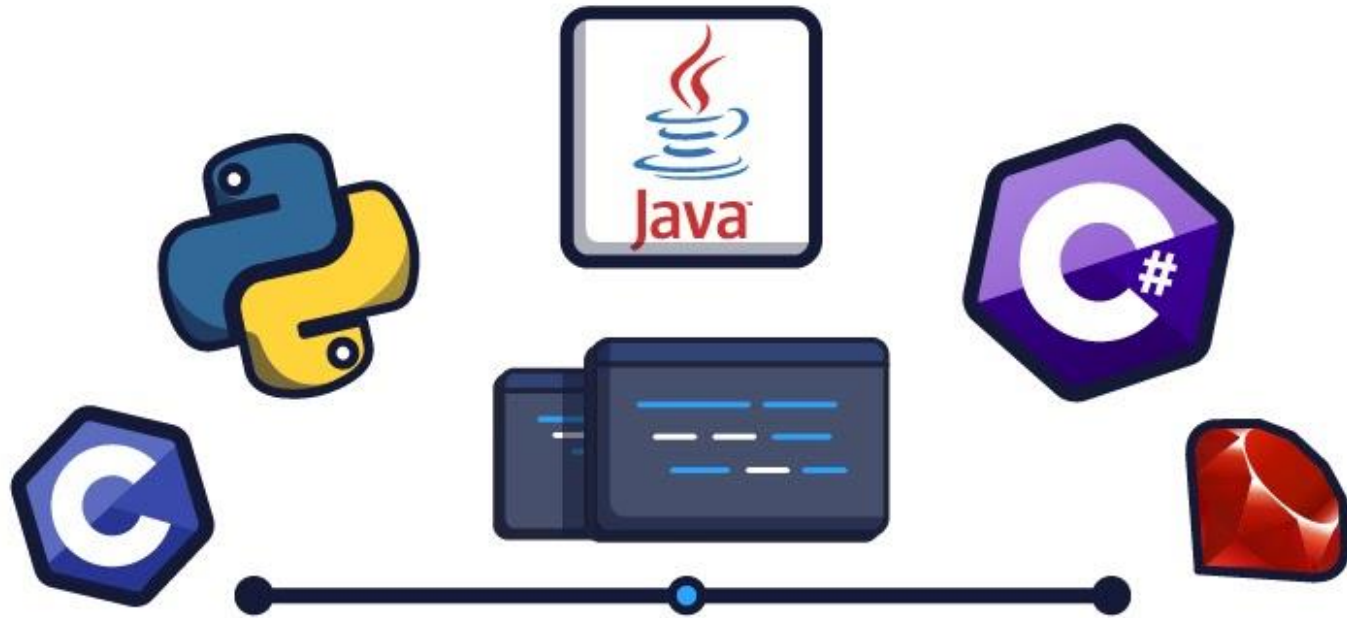
[1] Python is an interpreted, high-level, general-purpose programming language. Developers choose Python for a variety of use cases. Python applications range from game development to data science and machine learning.

4 Best OOP Languages



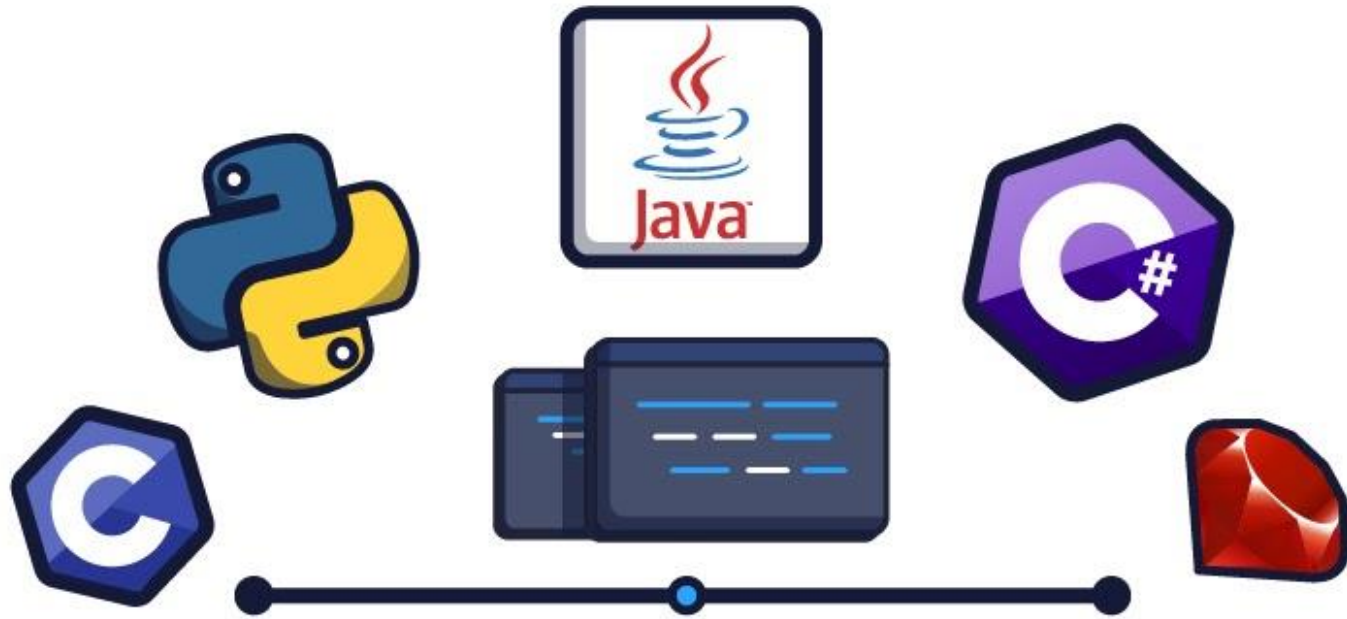
[2] As a class-based programming language, **Java** is designed to have few dependencies; thus, Java developers can look forward to uninterrupted reusability.

4 Best OOP Languages



C++, or 'C with Classes' is an object-oriented extension of C. C is a classic programming language that still ranks high in the TIOBE Index today. Yet its extension C++ performs exceptionally well when working with embedded systems like smartwatches and medical machines.

4 Best OOP Languages

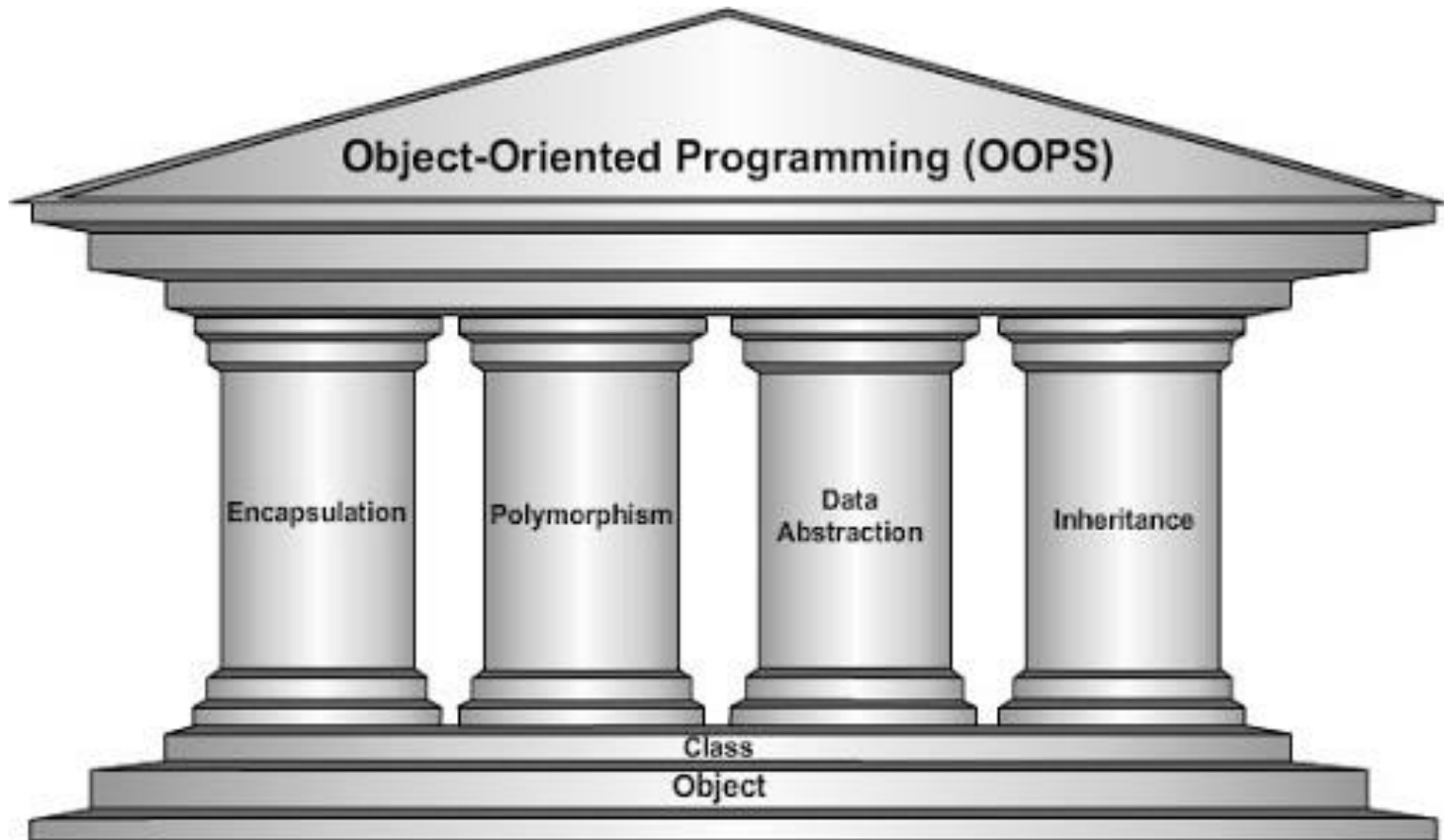


C# is a language of the .NET framework, a product of Microsoft that guides developers in building applications. Like C++, C# is also a middleware language that can work closely with hardware. C# is primarily used for game development in Unity.

A decorative border of green leaves and branches at the top of the slide.

1 - Basic principles (building blocks) *of OOP (object oriented programming)*

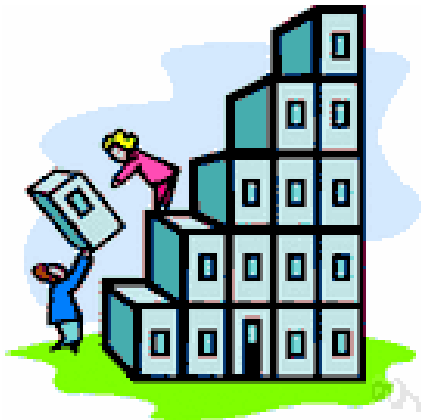
A decorative border of green leaves and branches at the bottom of the slide.



"***OOP in Python – Building Blocks***" dapat diartikan sebagai **konsep dasar** pemrograman berorientasi objek (OOP) dalam bahasa pemrograman Python, dengan fokus pada **elemen-elemen atau blok pembangunan utama**nya. Dalam hal ini, "*building blocks*" mengacu pada *basic concepts* yang membentuk dasar pemrograman berorientasi objek di Python.

Basic Principles (Building Blocks)

Jika kita menggambarkan sesuatu sebagai "*building blocks*", maka maksudnya adalah salah satu bagian terpisah yang digabungkan untuk membuat benda (*to make a thing*).



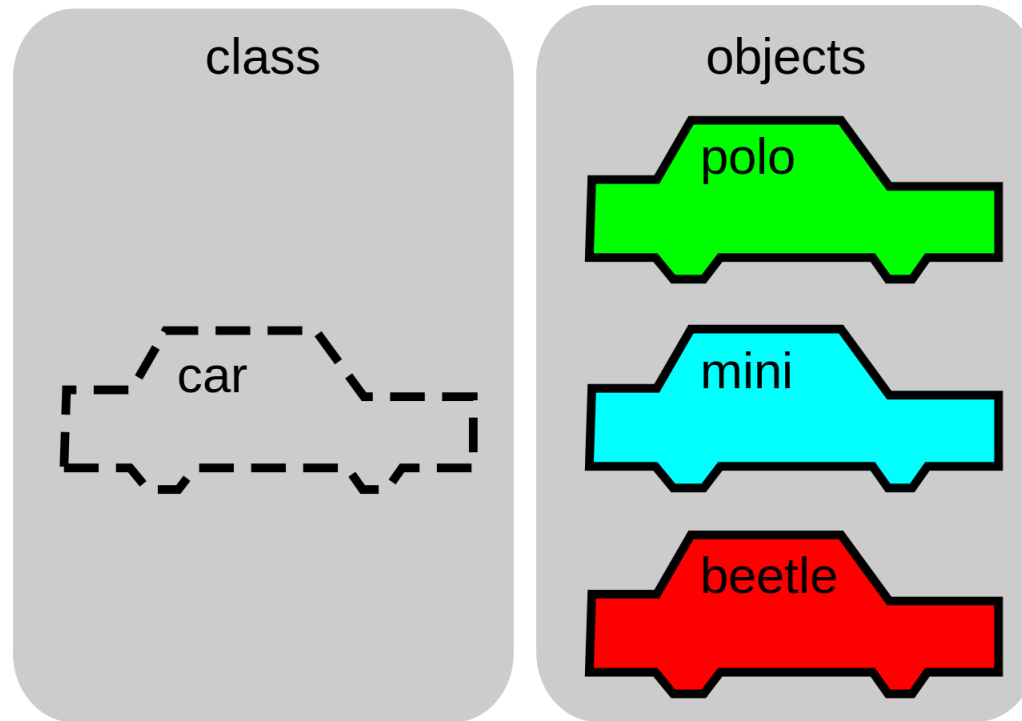
Object-oriented programming defines a mode (*style*) of programming that orients itself around structural objects with related properties (*variables and methods*).



What is Object-Oriented Programming?

- ❖ Object-oriented programming (OOP) is a **programming paradigm** that uses the concept of objects to construct well-defined, **manipulable pieces of code**.
- ❖ A **programming paradigm describes the way in which a program is organized**. Before the emergence of OOP, procedural and structural programming were the principal programming paradigms of the time.
- ❖ Procedural programming operates via **step-by-step instructions**. Structured programming is a bit more complex, **utilizing control flows like if/then/else statements and while/for loops**.
- ❖ However, object-oriented programming exerts the most leverage in a programming environment. With object-orienting programming, **we can generate a car class and code specificities of the object inside the class**.

What is Object-Oriented Programming?



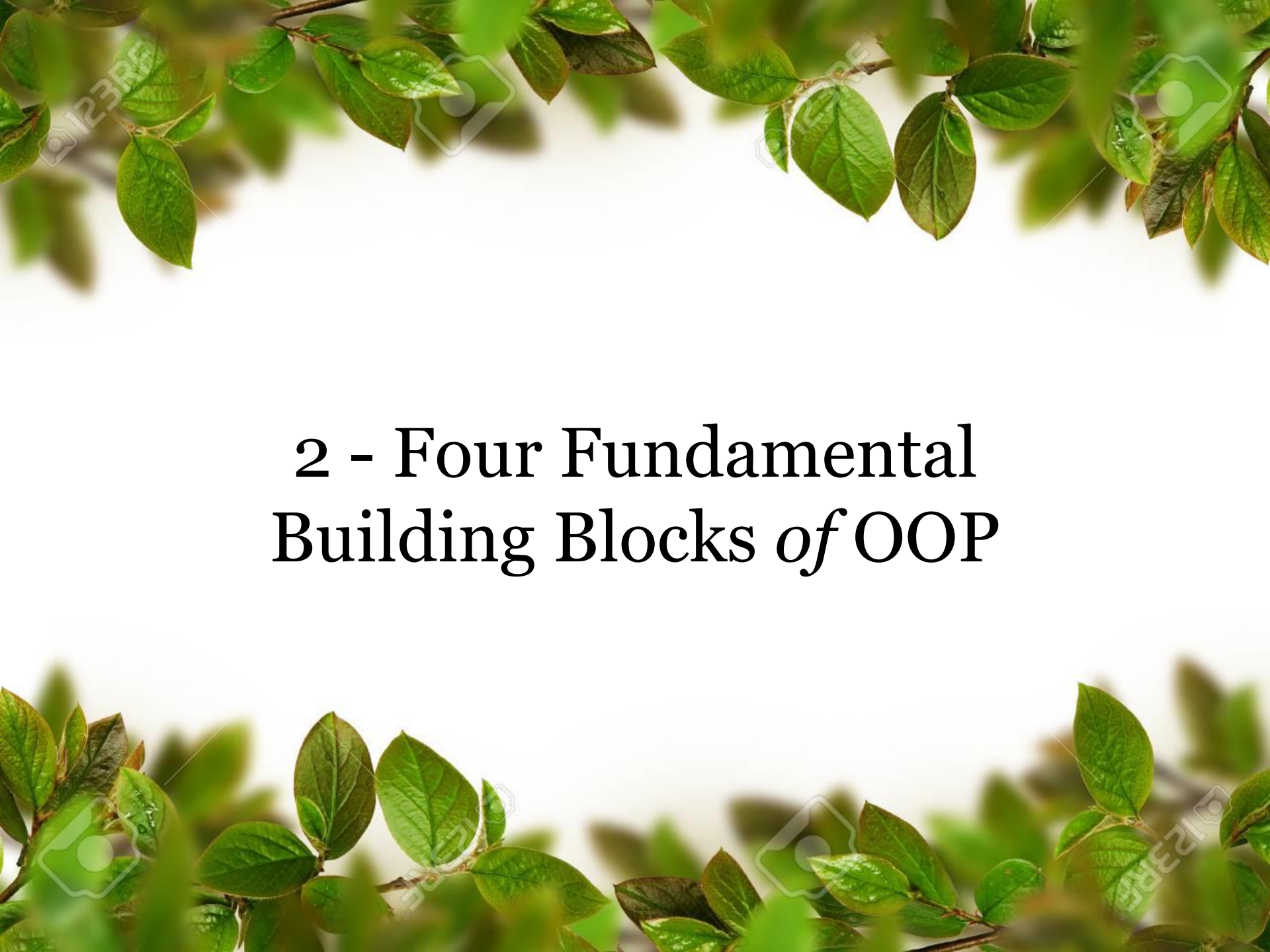
With object-orienting programming, **we can generate a car class and code specificities of the object inside the class.** Integral to the core of object-oriented programming is the **manipulation of objects.** **Objects contain data and code which hold properties, procedures, and functions.**

OOP Example



A car could be an *object* in Java.
Objects have *attributes* and *methods*.

Car Attributes		Car Methods	
Make & Model		Drive	
Nouns	Color	verbs	Park
	# of Doors		Reverse
	# of Seats		Tow



2 - Four Fundamental Building Blocks *of* OOP

4 Fundamental Building Blocks *of* OOP



Four fundamental building blocks (Basic Principles) cover/encompass **how object-oriented programming works:**

classes,

attributes,

methods, and

objects.

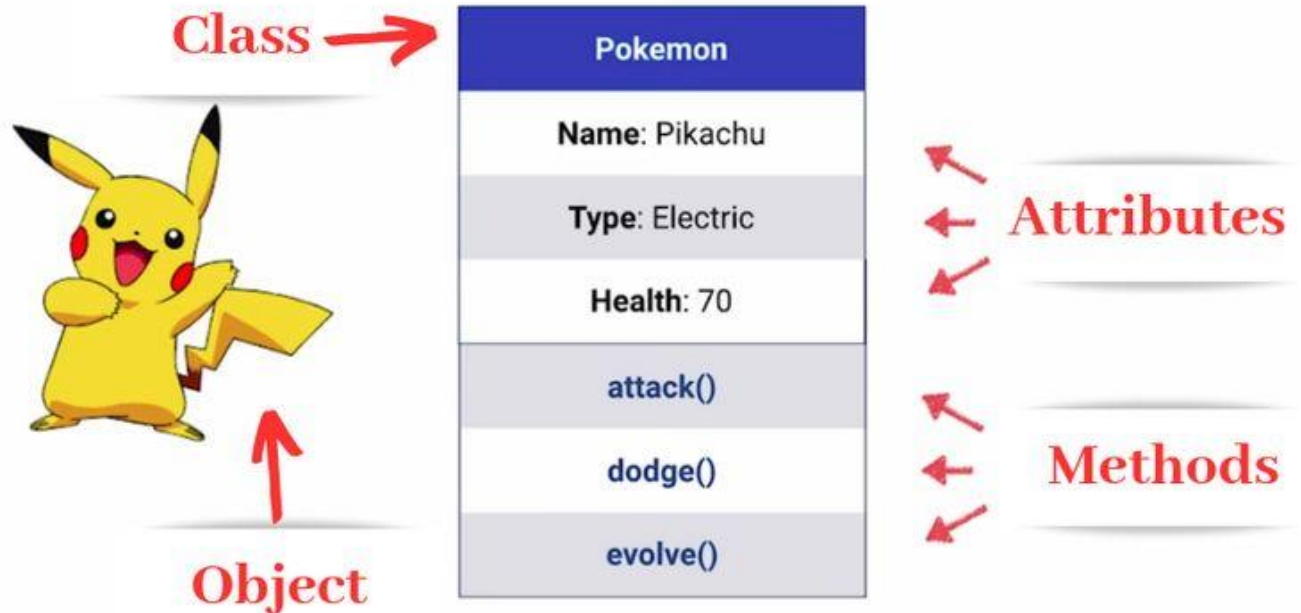
[1] Classes



Class



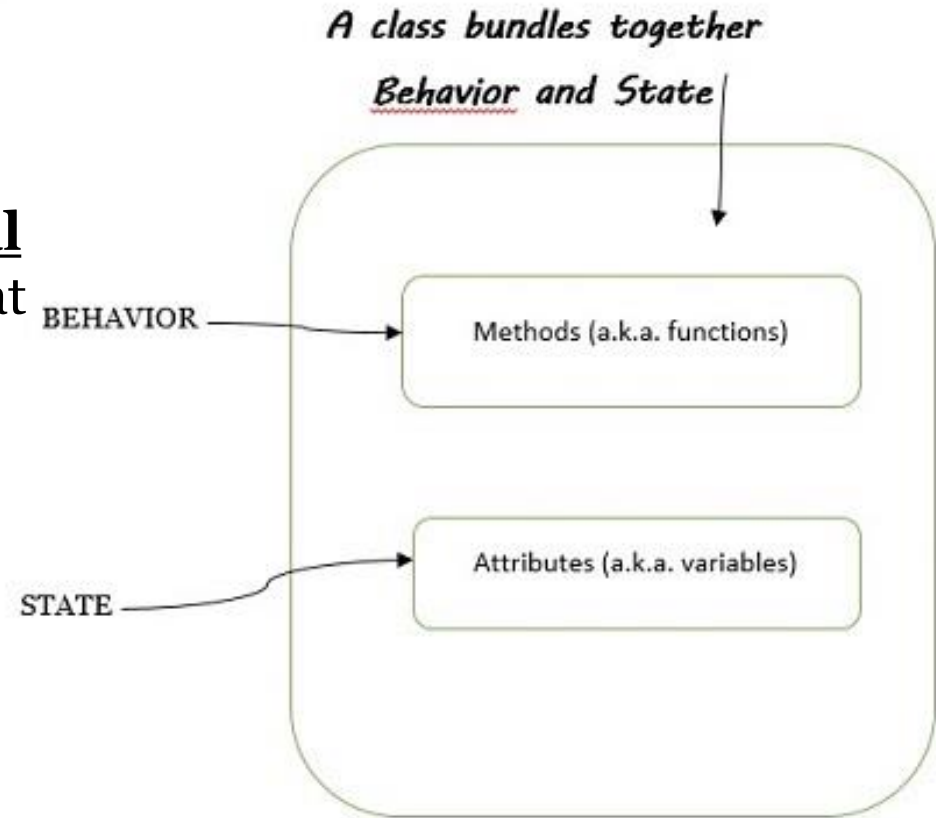
- ❖ Classes offer templates to better **characterize objects**. **In effect, classes serve as blueprints for generating objects.**
- ❖ Within a class, programmers must define the variables and methods that its corresponding **objects** can reference.
- ❖ Given the car example, a class would denote the properties of the car object, encase the car's functionality, as well as declare the car as a class in the first place.



Class Bundles: Behavior & State

[1] **The word behavior is identical to function** – it is a piece of code that does something (or implements a behavior).

[2] **The word state/attribute is identical to variables** – it is a place to store *values* within a *class*.



When we assert a class behavior and state together, it means that a class packages functions and variables.

[2] Attributes



- ❖ **Attributes (or variables) refer to the characteristics of the object.** Appearance, state, and other qualitative traits are common object attributes.
- ❖ **Class attributes** in combination with object instances differentiate objects from one another.

The following program demonstrates a class declaration in Python: class Car:

Constructor dengan parameter:

```
def __init__(self, color, type):
    self.color = color;
    self.type = type;
```

Keyword “self” represents an instance of the class for future referencing of the object’s attributes and methods. and ‘color’ and ‘type’ represent attributes of the class.

Classname (Identifier)	Student	Circle
Data Member (Static attributes)	name grade	radius color
Member Functions (Dynamic Operations)	getName() printGrade()	getRadius() getArea()

SoccerPlayer	Car
name number xLocation yLocation	plateNumber xLocation yLocation speed
run() jump() kickBall()	move() park() accelerate()

Examples of classes

1

```
class Student:

    # Constructor
    # initialize instance variable
    def __init__(self, name):

        print('Inside constructor')
        self.name = name
        print('All variables initialized')

    # Instance Method
    def show(self):
        print('Hello, my name is', self.name)

# Create object using constructor
s1 = Student('Alex')
s1.show()
```

```
Inside constructor
All variables initialized
Hello, my name is Alex
```

2

```
1 class GreetClass:
2     def __init__(self):
3         self.say_hello = "Hello!"
4         self.say_hi = "Hi!"
5
6     hello_obj = GreetClass()
7     print(hello_obj.say_hello)
8
9     hi_obj = GreetClass()
10    print(hi_obj.say_hi)
```

Run: main

C:\Users\ak111\PycharmProjects\pythonProject\venv\Scripts\python.exe

Hello!

Hi!

3

```
1 class Line:
2     def __init__(self, p1, p2):
3         self.line = (p1, p2)
4
5     def what(self):
6         print('Object is a line')
7
8 class Dot:
9     def __init__(self, p1):
10        self.point = p1
11
12    def what(self):
13        print('Object is a dot')
14
```

**Pilihlah sebuah
class yang memiliki
Constructor tanpa
parameter ???**

[3] Methods

Classname (Identifier)	Student	Circle
Data Member (Static attributes)	name grade	radius color
Member Functions (Dynamic Operations)	getName() printGrade()	getRadius() getArea()

SoccerPlayer	Car
name number xLocation yLocation	plateNumber xLocation yLocation speed
run() jump() kickBall()	move() park() accelerate()

Examples of classes

Programmers must define methods alongside attributes. **Methods encapsulate functions that handle the data and behavior of an object instance.**

In a car class, a drive method might be appropriate. We can define such a method right below the car's attribute definitions.

[4] Objects



Objects exist alongside classes. They are essentially data fields with distinct structures that the programmer can determine.

Once we call an object, the program creates an instance. An instance is a specific object generated from a class.

Object – one of the instances of the class, perform functionality defined in the class.

```

class.php
1  <?php
2  class calculation{
3      public $a ,$b ,$c;      //properties
4
5      function sum(){          //methods
6          $this->c=$this->a + $this->b;
7          return $this->c;
8      }
9      function sub(){
10         $this->c=$this->a - $this->b;
11         return $this->c;
12     }
13 }

```

```

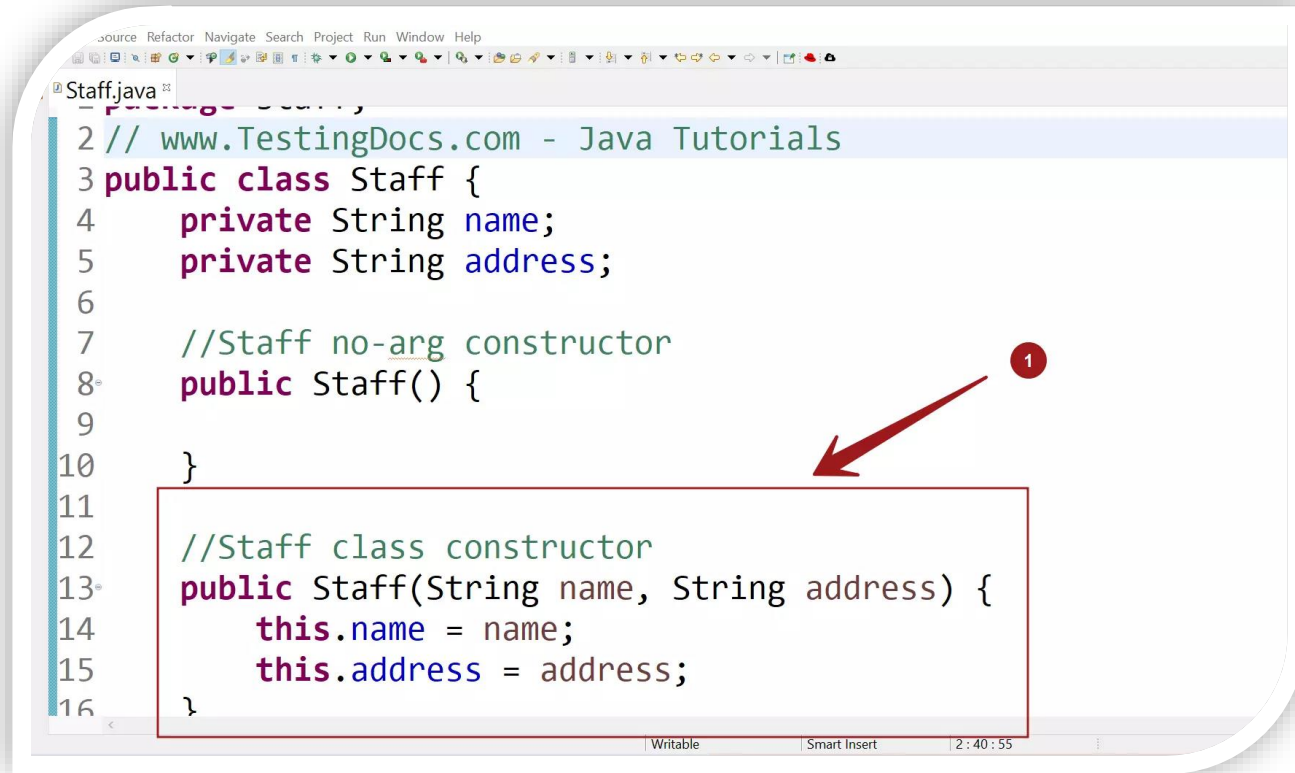
class CrawledArticle():
    def __init__(self, title, emoji,
        self.title = title
        self.emoji = emoji
        self.content = content
        self.image = image

class ArticleFetcher():
    def fetch(self):
        r = requests.get("http://pyth
        doc = BeautifulSoup(r.text, "

```

Di dalam sebuah *class* OOP, contohnya Bahasa pemrograman PHP/Java menggunakan “**this keyword**”, sedangkan Python menggunakan “**self keyword**”. Kegunaannya tetap sama, yaitu digunakan untuk merujuk pada objek saat ini dalam **class OOP** dan memungkinkan manipulasi properti atau pemanggilan metode objek tersebut.

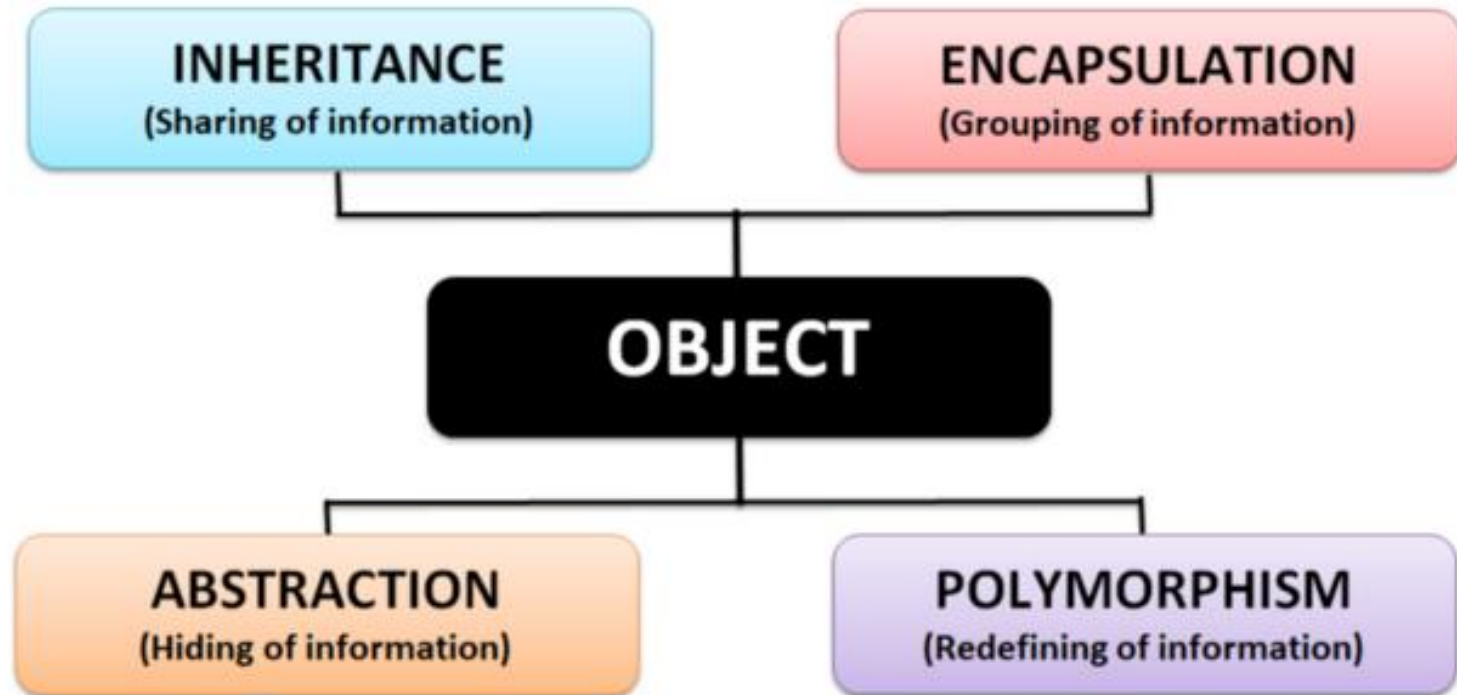
Bahasa Pemrograman Java



```
Staff.java
2 // www.TestingDocs.com - Java Tutorials
3 public class Staff {
4     private String name;
5     private String address;
6
7     //Staff no-arg constructor
8     public Staff() {
9
10    }
11
12    //Staff class constructor
13    public Staff(String name, String address) {
14        this.name = name;
15        this.address = address;
16    }
```

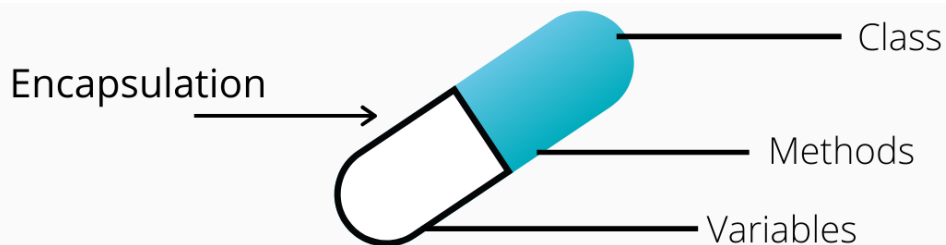
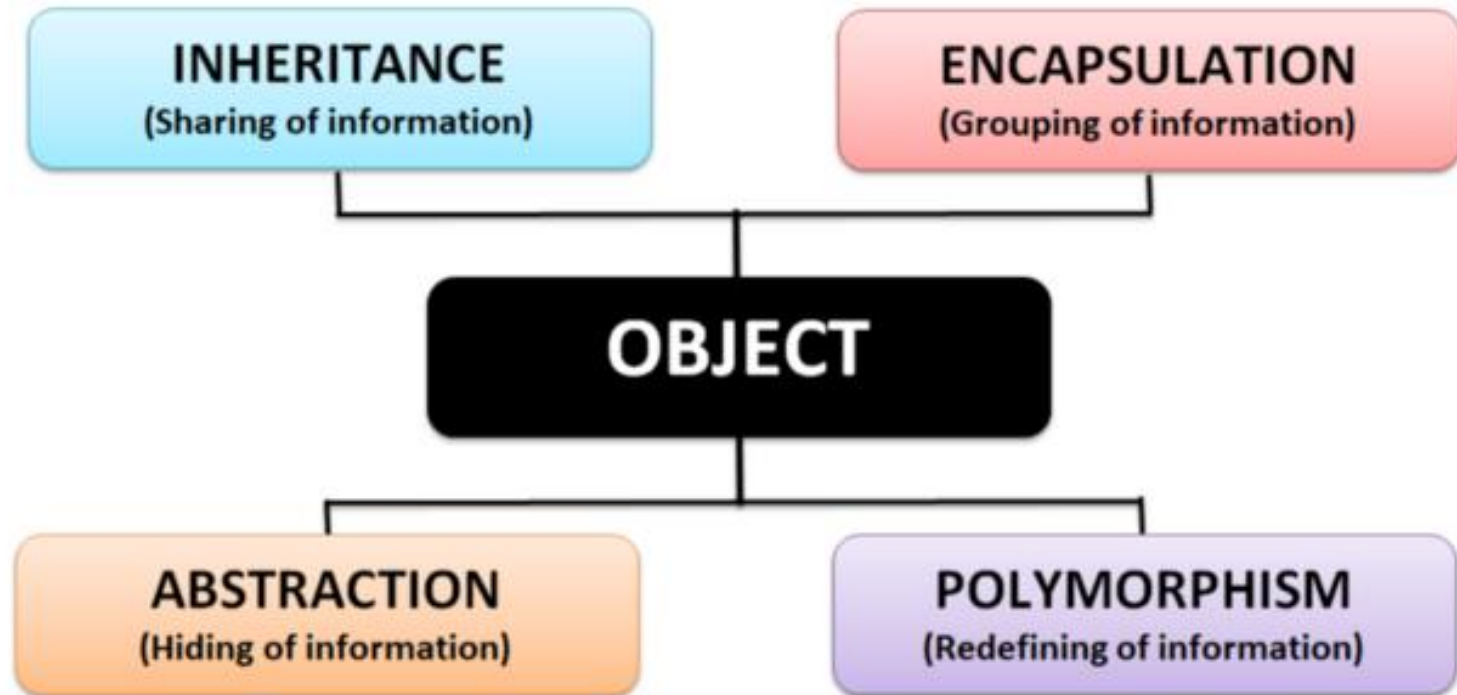
Jika terdapat konflik antara “**nama parameter method**” dan “**variabel instans**”, maka “this keyword” dapat digunakan untuk membedakan keduanya. “this keyword” juga dapat digunakan untuk memanggil *method* dari objek yang sedang digunakan.

Four Pillars of OOP



In addition to the fundamental building blocks of object-oriented programming, the following technical principles are critical to any object-oriented programming approach.

Four Pillars of OOP



Encapsulation is an Object Oriented Programming Concept that binds together the data and functions that manipulate the data, and that keeps both safe from outside interference and misuse

Benefits of Object-Oriented Programming



What Are the
Benefits of
Object-Oriented
Programming?

Object-oriented programming is good (favorable) because of its malleability (*sifat lunak*). The ability to regard structures of code as objects in a programming environment empowers programmers to shape programs freely.

```
31 def __init__(self, path):
32     self.file = None
33     self.fingerprints = set()
34     self.logdups = True
35     self.debug = debug
36     self.logger = logging.getLogger(__name__)
37     if path:
38         self.file = open(os.path.join(path, 'requests.log'),
39                         'a')
40         self.file.seek(0)
41         self.fingerprints.update(self._get_fingerprints())
42
43 @classmethod
44 def from_settings(cls, settings):
45     debug = settings.getbool('DEBUG', False)
46     return cls(job_dir(settings), debug)
```

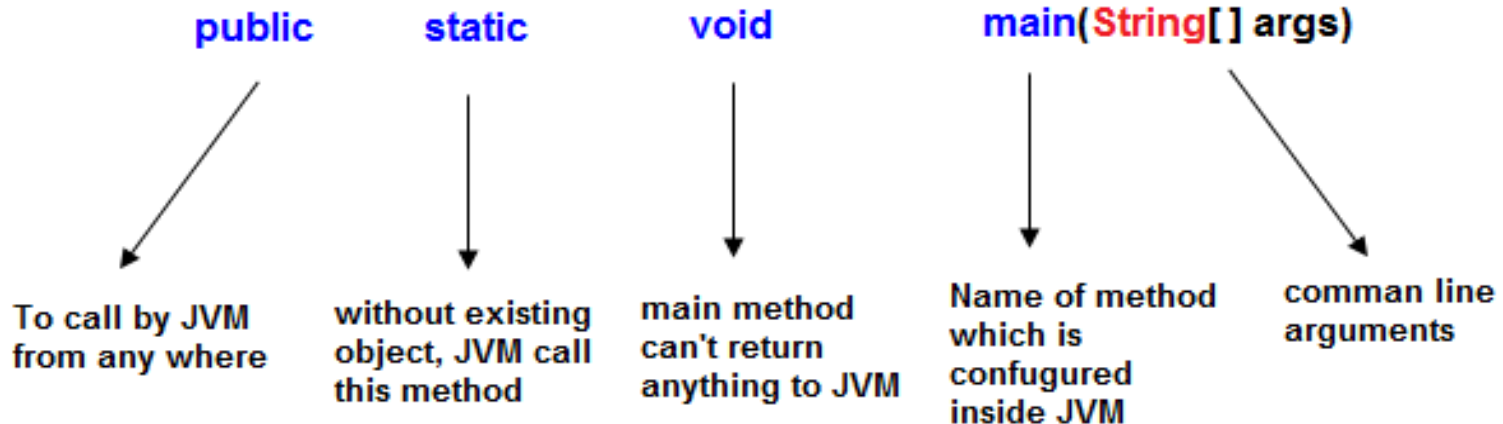
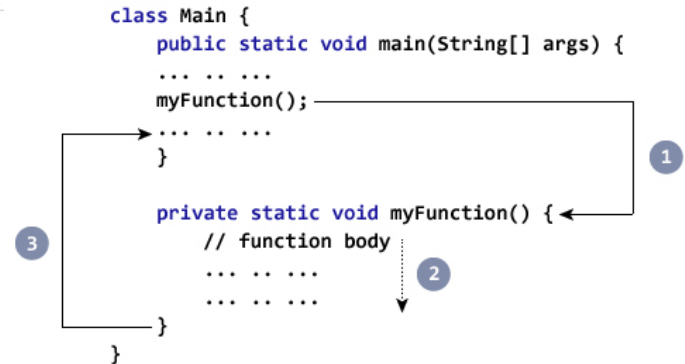
By understanding the basic concepts of OOP using Python, we as students can build programs that are more **structured, modular** (*kemampuan untuk membagi program menjadi bagian-bagian yang lebih kecil yang disebut modul atau kelas*), and **easy to understand**, thus improving the readability (*keterbacaan*) and maintainability (*pemeliharaan*) of the code.



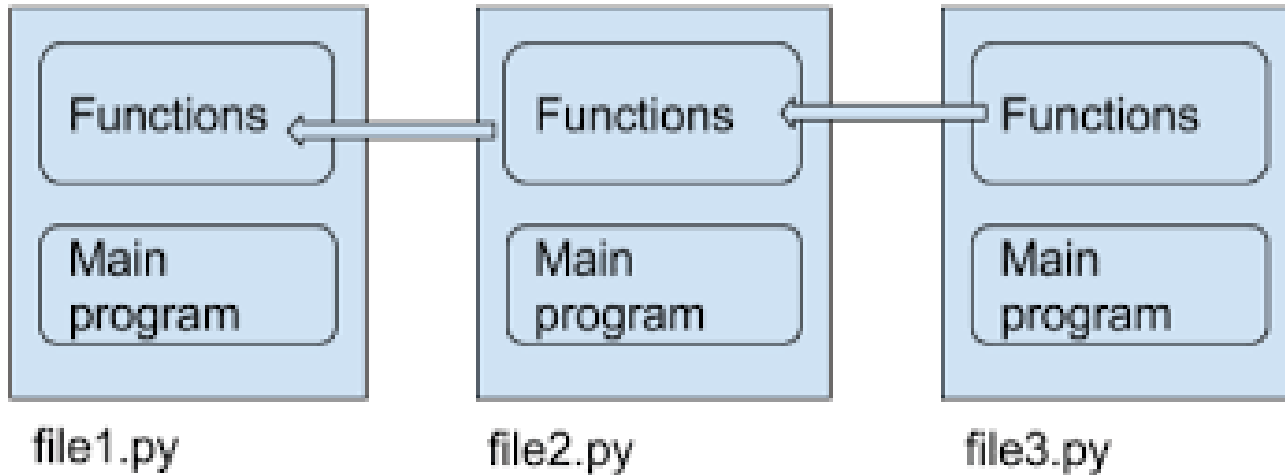
Exercise *for* Students

Main Functions in Java

```
1 public class MethodsExample{
2     //Creating a Method
3     public int addNumbers(int x, int y){
4         int addition = x + y;
5         return addition;
6     }
7     public static void main(String args[]){
8         int i=10;
9         int j=25;
10        MethodsExample obj = new MethodsExample();
11        int result = obj.addNumbers(i, j);
12        System.out.println("Sum of x+y = " + result);
13    }
14 }
15
```

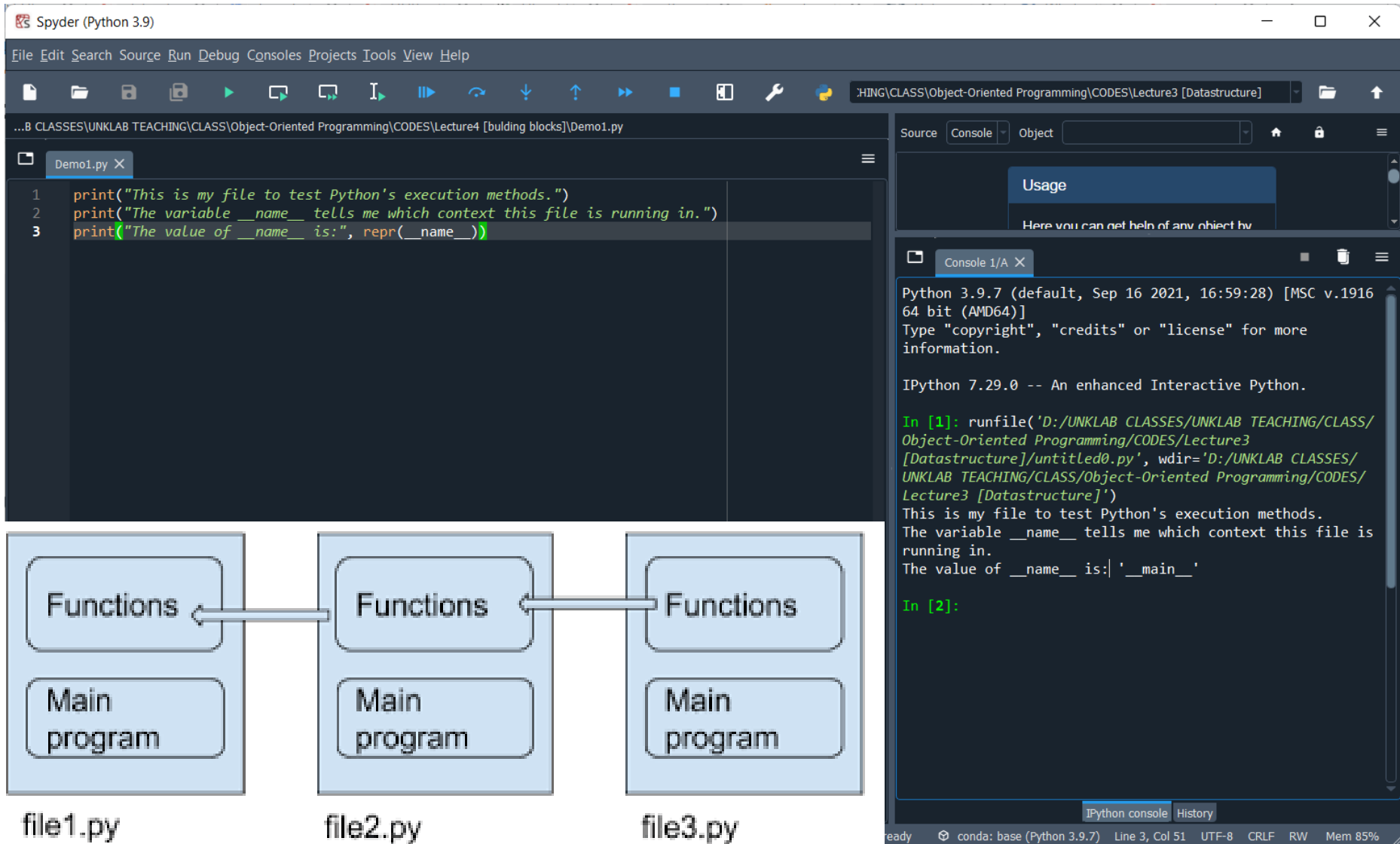


Special Variable Called “__name__”



- ❖ Python defines a special variable called `__name__` that contains a string whose value depends on how the code is being used.
- ❖ The `__name__` variable (two underscores before and after) is a special Python variable. It gets its value depending on how we execute the containing script (*main file*). Sometimes we write a script with functions that might be useful in other scripts as well. In Python, we can import that script as a module in another script.
- ❖ Make a conditional (or if) statement that checks the value of `__name__` and compares it to the string `"__main__"`.

Main File is Current Executed File



The screenshot shows the Spyder Python IDE interface. The main editor displays a file named `Demo1.py` with the following code:

```
1 print("This is my file to test Python's execution methods.")
2 print("The variable __name__ tells me which context this file is running in.")
3 print("The value of __name__ is:", repr(__name__))
```

The console output shows the execution of the code:

```
Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916
64 bit (AMD64)]
Type "copyright", "credits" or "license" for more
information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/
Object-Oriented Programming/CODES/Lecture3
[Datastructure]/untitled0.py', wdir='D:/UNKLAB CLASSES/
UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/
Lecture3 [Datastructure]')
This is my file to test Python's execution methods.
The variable __name__ tells me which context this file is
running in.
The value of __name__ is:| '__main__'

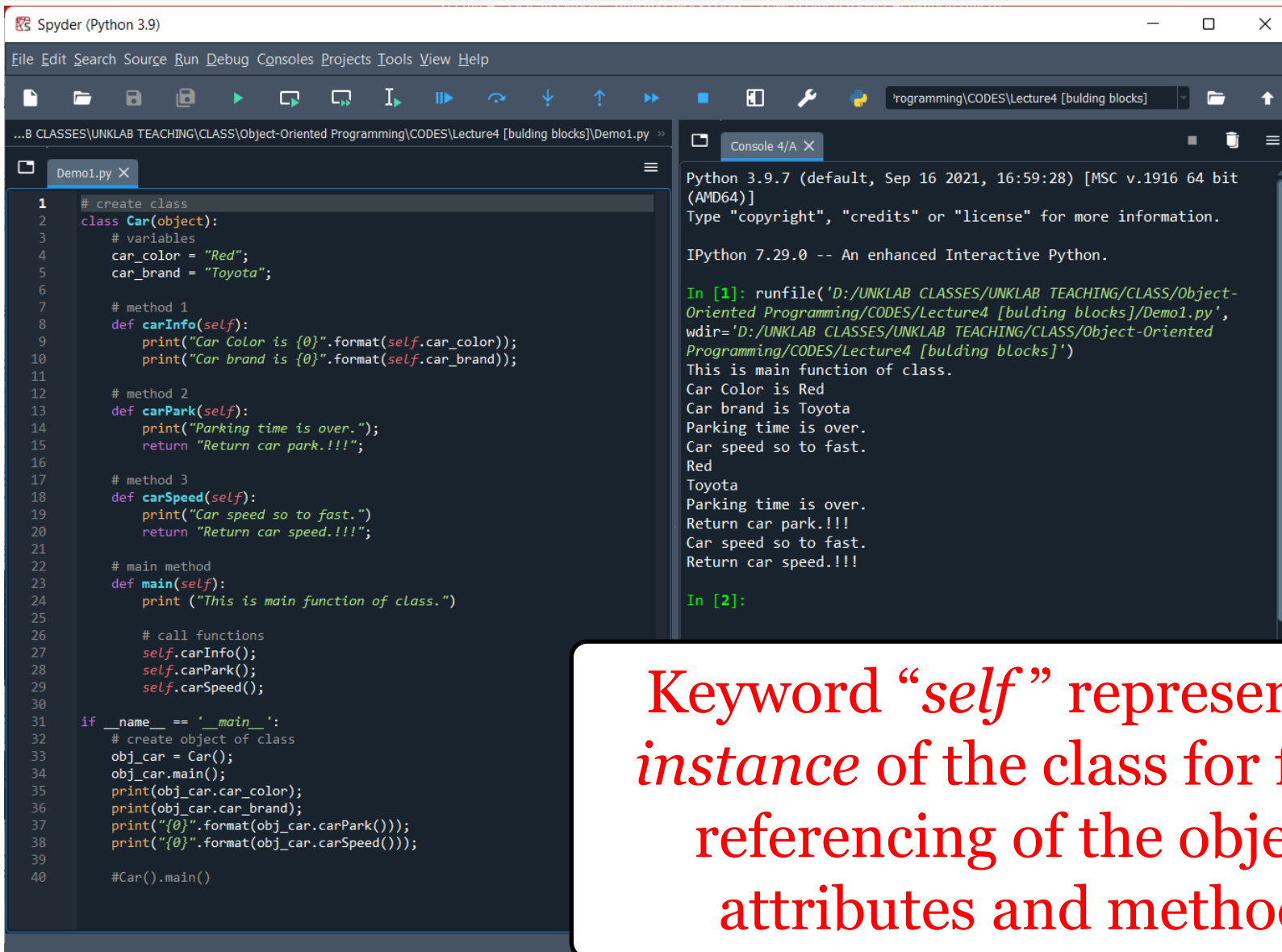
In [2]:
```

Below the IDE screenshot, a diagram illustrates the execution flow between three files:

```
graph LR
    subgraph file1_py [file1.py]
        F1[Functions]
        MP1[Main program]
    end
    subgraph file2_py [file2.py]
        F2[Functions]
        MP2[Main program]
    end
    subgraph file3_py [file3.py]
        F3[Functions]
        MP3[Main program]
    end
    MP1 --> F2
    MP2 --> F3
```

The diagram shows three boxes representing files: `file1.py`, `file2.py`, and `file3.py`. Each box contains two sub-components: `Functions` and `Main program`. Arrows indicate the execution flow: from the `Main program` of `file1.py` to the `Functions` of `file2.py`, and from the `Main program` of `file2.py` to the `Functions` of `file3.py`.

Main Function Demo



The screenshot displays the Spyder Python IDE interface. The left pane shows a Python script named `Demo1.py` with the following code:

```
1 # create class
2 class Car(object):
3     # variables
4     car_color = "Red";
5     car_brand = "Toyota";
6
7     # method 1
8     def carInfo(self):
9         print("Car Color is {}".format(self.car_color));
10        print("Car brand is {}".format(self.car_brand));
11
12    # method 2
13    def carPark(self):
14        print("Parking time is over.");
15        return "Return car park.!!!";
16
17    # method 3
18    def carSpeed(self):
19        print("Car speed so to fast.");
20        return "Return car speed.!!!";
21
22    # main method
23    def main(self):
24        print ("This is main function of class.")
25
26    # call functions
27    self.carInfo();
28    self.carPark();
29    self.carSpeed();
30
31 if __name__ == '__main__':
32     # create object of class
33     obj_car = Car();
34     obj_car.main();
35     print(obj_car.car_color);
36     print(obj_car.car_brand);
37     print("{} {}".format(obj_car.carPark(), obj_car.carSpeed()));
38     print("{} {}".format(obj_car.carPark(), obj_car.carSpeed()));
39
40 #Car().main()
```

The right pane shows the console output for the script:

```
Python 3.9.7 (default, Sep 16 2021, 16:59:28) [MSC v.1916 64 bit (AMD64)]
Type "copyright", "credits" or "license" for more information.

IPython 7.29.0 -- An enhanced Interactive Python.

In [1]: runfile('D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture4 [bulding blocks]/Demo1.py',
wdir='D:/UNKLAB CLASSES/UNKLAB TEACHING/CLASS/Object-Oriented Programming/CODES/Lecture4 [bulding blocks]')
This is main function of class.
Car Color is Red
Car brand is Toyota
Parking time is over.
Car speed so to fast.
Red
Toyota
Parking time is over.
Return car park.!!!
Car speed so to fast.
Return car speed.!!!

In [2]:
```

Keyword “*self*” represents an *instance* of the class for future referencing of the object’s attributes and methods.

Modifikasi source code pada file Demo1.py:

1. Dalam file **Demo1.php**, kata “*object*” pada kata kunci “**class Car(object):**” sebenarnya tidak diperlukan dalam Python versi 3.0 ke atas. Karena, semua *class* secara default mengambil “*object*” sebagai class dasar jika tidak diberikan. Ubah code tersebut menjadi “**class Car:**”
2. Tambahkan **constructor tanpa parameter** pada class Car.
3. Tambahkan sebuah variable “***car_year***” dan isi dengan value “Tahun 2023”.
4. Buat method ke-5 dari class Car, nama method yaitu “***carStatus***”. Method tersebut dapat mengembalikan nilai berupa String.
5. Tampilkan output dari variable “***car_year***” menggunakan object dari class Car.
6. Tampilkan output dari method “***carStatus***” menggunakan object dari class Car.

Submit Report

Submit class **Lab Exercise #1 (Lecture4)** berikut ini di GCR, mahasiswa diminta untuk menjelaskan structure dari class yang dibuat. Screenshot code dan output program disertai penjelasan.

END PRESENTATION

Thank you for your attention

Instructor: S – W – T

THANK YOU

